## 1 Laden von Konstanten in Register

*Anw-01* Kleine Konstanten können direkt im Maschinenwort untergebracht werden.

*Anw-02* Registerinhalte (im Hex-Format) nach Ausführung von Anw-02:

**Register R0**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| 00 | 00 | 00 | 12 |

**Register R1**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| ff | ff | ff | 80 |

*Anw-03* Größere Konstante erfordern einen zusätzlichen Speicherplatz zum Speichern des Wertes. Dieser kann automatisch vom Assembler erzeugt werden (Gleichheitszeichen). Durch Auswertung des Disassemblers kann die Adresse des Speicherplatzes bestimmt werden:

**Adresse der Konstanten:**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| 08 | 00 | 02 | 30 |

**Inhalt Register R2 nach Anw-03:**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| 12 | 34 | 56 | 78 |

Zum Zugriff auf den zusätzlichen Speicherplatz setzt der Assembler eine besondere Assembleranweisung ein. Geben Sie an, welche Anweisung die CPU tatsächlich ausführt:

ldr r2, [pc, #56]

## 2 Zugriff auf Variable

*Anw-04* Variable werden im Datenbereich des Prozessors untergebracht. Auf die Variablen kann nicht direkt zugegriffen werden, sondern es muss

*Anw-05* zuvor deren Speicheradresse in ein Register geladen werden:

**Adresse v. VariableA (Reg. R0 nach Anw-05)**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| 20 | 00 | 00 | 0C |

**Inhalt Register R1 nach Anw-05:**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| 00 | 00 | 12 | 34 |

**Anzahl von Anw-05 geladener Bits:** 16

*Anw-06* Der Assembler überprüft nicht, ob der Lade/Speicherbefehl mit der Definition der Variable übereinstimmt. Anw-06 lädt 32 Bits, obwohl VariableA nur 16 Bit umfasst:

**Inhalt Register R2 nach Anw-06:**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| 47 | 11 | 12 | 34 |

**Anzahl von Anw-06 geladener Bits:** 32

*Anw-07* Die Adresse einer Variable kann auch als Differenz zu einer Basis angegeben werden:

**Adresse v. VariableC:**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| 20 | 00 | 00 | 10 |

**Inhalt von VariableC nach Anw-07:**

| 31 … 24 | 23 … 16 | 15 … 8 | 7 … 0 |
|---|---|---|---|
| 47 | 11 | 12 | 34 |

**Anzahl von Anw-07 gespeicherter Bits:** 32

## 3 Zugriff auf Felder

Die Elemente eines Feldes sind alle vom gleichen Typ und werden der Reihe nach im Speicher angelegt.

| | Anfangsadresse: | Anzahl Elemente: | Anzahl Bytes pro Element: |
|---|---|---|---|
| MeinHalbwortFeld: | 20 00 00 14 (31 … 16 15 … 0) | 6 | 1 |
| MeinWortFeld: | 20 00 00 20 (31 … 16 15 … 0) | 6 | 4 |
| MeinTextFeld: | 20 00 00 38 (31 … 16 15 … 0) | 9 | 1 |

*Anw-08* Der Zugriff auf ein bestimmtes Element kann durch das Laden der Anfangsadresse in ein Register und die Angabe der Adressdifferenz für
*Anw-09* das jeweilige Element erfolgen.
*Anw-10*

| Register R1 nach Anw-09: | Register R2 nach Anw-10: |
|---|---|
| 00 00 00 22 (31 … 16 15 … 0) | 00 00 00 3c (31 … 16 15 … 0) |

*Anw-11* Die Adressdifferenz kann auch über ein Register angegeben werden. Sie kann somit während des Programmlaufs verändert werden:
*Anw-12*

| Register R4 nach Anw-12: | Nr. (beginnend mit 0) des von Anw-12 gelesenen Elements: |
|---|---|
| 00 00 00 45 (31 … 16 15 … 0) | 5 |

*Anw-13* Die Basisadresse kann automatisch nach Zugriff auf ein Element erhöht werden.

| Register R0 nach Anw-13 | Register R5 nach Anw-13: | Nr. (beginnend mit 0) des von Anw-13 gelesenen Elements: |
|---|---|---|
| 20 00 00 16 (31 … 16 15 … 0) | 00 00 00 3c (31 … 16 15 … 0) | 1 |

*Anw-14*

| Register R0 nach Anw-14 | Register R6 nach Anw-14: | Nr. (beginnend mit 0) des von Anw-14 gelesenen Elements: |
|---|---|---|
| 20 00 00 18 (31 … 16 15 … 0) | 00 00 ff cc (31 … 16 15 … 0) | 2 |

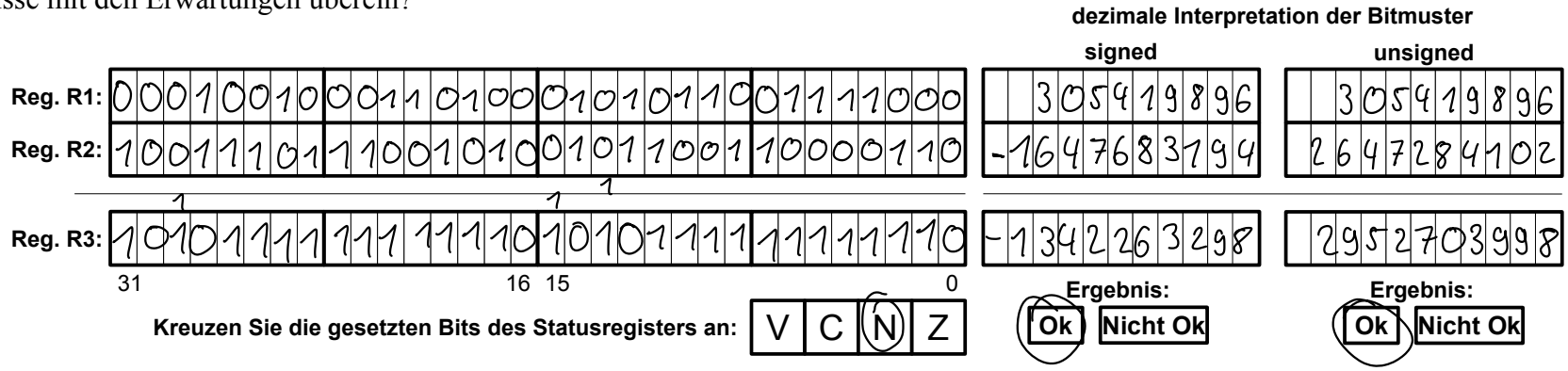
*Anw-15*

| Register R0 nach Anw-15 | Adresse des von Anw-15 überschriebenen Elements: | Nr. (beginnend mit 0) des von Anw-15 überschriebenen Elements: |
|---|---|---|
| 20 00 00 1a (31 … 16 15 … 0) | 20 00 00 1a (31 … 16 15 … 0) | 31 |

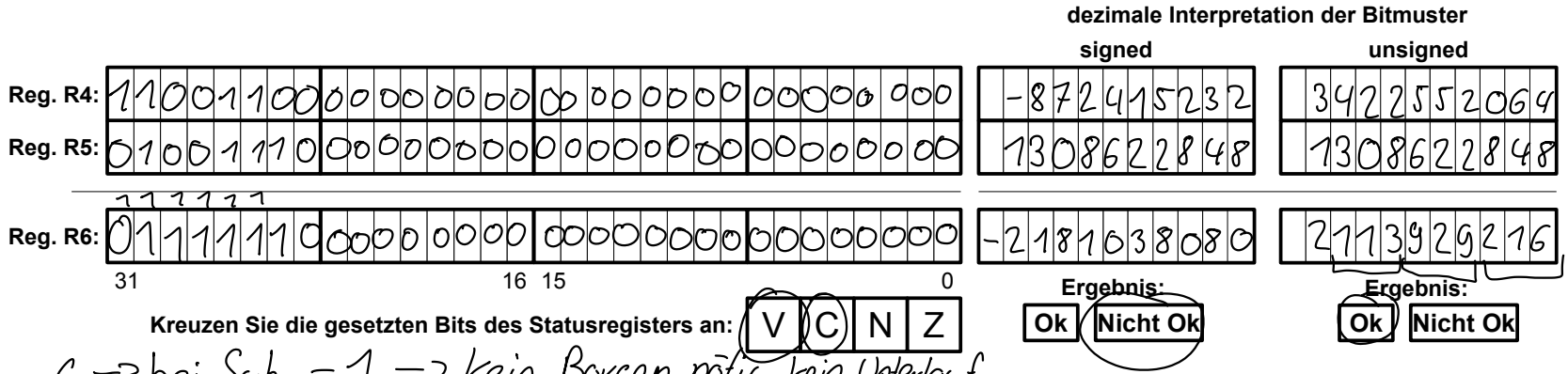
## *4 Addition und Subtraktion von vorzeichenlosen und vorzeichenbehafteten Integer-Werten*

Arithmetik-Befehle haben 3 Operanden. Tragen Sie in das nachfolgende Schema die beteiligten Werte in binärer Form ein. Wandeln Sie die binäre Darstellungen um in vorzeichenbehaftete und vorzeichenlose Dezimalzahlen. Überprüfen Sie die Rechnung. Stimmen die Ergebnisse mit den Erwartungen überein?

*Anw-16*
*Anw-17*
*Anw-18*
*Anw-19*

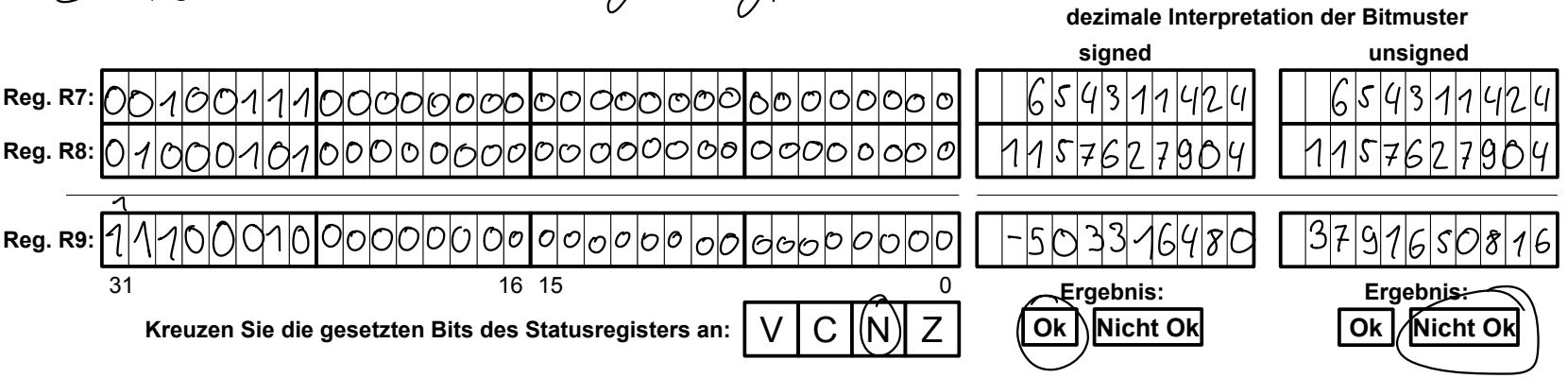



*Anw-20*
*Anw-21*
*Anw-22*




C → bei Sub. = 1 → kein Borgen nötig, kein Unterlauf

*Anw-23*
*Anw-24*
*Anw-25*



R2: 10011101110010100101100110000110

01100010011010110100111001

0110 0010 0011 0101 1010 0110 0111 1001+1

= 0110 0010 0011 0101 1010 0110 0111 1010

=

1100|1100|0000|0000|0000|0000|0000|0000

0011|0011|1111|1111|1111|1111|1111|1111 +1

0011 0100 0000 0000 0000 0000 0000 0000